泰坦尼克号生还预测 —— 学习笔记

PyTorch 二分类建模全流程 | 数据集：train. csv（891 条）

一、任务理解与整体思路

这道题本质上是一个二分类问题：给一名乘客的个人信息，判断他最终是幸存（1）还是遇难（0）。题目明确要求必须用 PyTorch 自己搭模型、自己写训练循环，不能拿现成的分类器（比如 sklearn 里的随机森林）直接顶替，所以我的思路是：先把表格数据处理成数字矩阵，然后自己定义一个简单的全连接神经网络，用 BCELoss 算误差、用 Adam更新参数，跑若干轮之后在测试集上看准确率。

整个流程拆成四步：① 数据清洗和特征处理 → ② 划分数据集并封装成 DataLoader → ③ 定义模型、损失函数、优化器并训练 → ④ 保存模型并对新乘客做预测。对应到代码就是 data_process. py、model. py、train. py、predict. py 四个文件，main. py 是一键运行入口。

二、数据清洗与特征处理

原始 csv 有 12 个字段，但不是每个都有用。我的取舍如下：

字段	处理方式	理由
PassengerId	丢弃	只是编号，和生还与否没有任何关系
Name	丢弃	纯文本，本次不做文本特征提取
Ticket	丢弃	票号字符串，规律性弱，难以直接利用
Cabin	丢弃	缺失约 77%，强行补全反而引入噪声
Age	中位数填充	有缺失；年龄分布有偏，中位数比均值稳健
Fare	中位数填充	少量缺失，同样用中位数
Embarked	众数填充	类别型字段，填出现最多的港口 S
Pclass	保留 +one-hot	舱位等级和生还率强相关，是重要特征
Sex	保留 +one-hot	题目已提示性别影响很大
SibSp / Parch	保留 + 构造新特征	家庭规模会影响逃生决策
Survived	作为标签 y	不参与特征矩阵

1）缺失值为什么用中位数而不是均值

票价 Fare 里有极少数特别贵的票（最高 512），年龄也有个别很大的值。如果用均值填充，这些极端值会把平均值往上拉，填出来的数就不代表“普通人”了。中位数不受极端值影响，所以更稳。Embarked 是类别，只能填出现次数最多的那个值。

2）特征工程：家庭规模 FamilySize

我新造了一个特征 FamilySize =SibSp +Parch + 1（加 1 是算上本人）。直觉上，独自一人的人和拖家带口的人，逃生时的处境完全不同：一个人跑得快，一家人可能会因为要互相等待而耽误。把 SibSp 和 Parch 合起来，模型更容易学到这个规律。我没有再额外做 IsAlone（是否独自一人），因为它和 FamilySize == 1 完全等价，属于重复信息。

3）数值标准化与类别 one-hot

神经网络对输入尺度很敏感。Fare 的数值能到几百，而 Parch 通常只有 0~2，如果不处理，模型会被大数值的特征“带偏”，训练也很难收敛。所以我对数值特征做标准化：(x - 均值) / 标准差，把它们压到 0 附近。类别特征（Pclass / Sex / Embarked）是文字或编号，不能直接当数字算，所以做了 one-hot 展开：Sex 变成 Sex_female、Sex_male 两列，对应的填 1、其余填 0。

最终得到 13 个特征：Age、Fare、SibSp、Parch、FamilySize（5 个数值）+Pclass_1/2/3（3 个）+ Sex_female、Sex_male（2 个）+Embarked_C/Q/S（3 个）。

三、数据划分、随机种子与可复现性

我采用 8 ：2 的比例划分：891 条里 713 条训练、178 条测试。划分方式是先用 np. random. permutation 把下标随机打乱，再切前 20%当测试集，没有用 sklearn 的现成接口。

随机种子统一设为 SEED= 42，在 numpy 和torch 上都设了一遍，这样数据怎么打乱、模型参数怎么初始化、每批数据怎么 shuffle 都是确定的，别人拿到代码再跑一次能得到完全相同的结果。

一个容易踩的坑：预处理参数必须先划分再算

标准化要用到均值和标准差。如果先拿全部 891 条算出均值再划分，那么测试集的信息就“泄露”进预处理里了，最后测出来的准确率是虚高的。所以我的顺序是：先划分 → 只在训练集（713 条）上算参数 → 用同一套参数去转换测试集和新乘客。这些参数最后会存进 preprocess_params. pkl，预测时直接读出来复用。

四、模型结构、损失函数与优化器

模型是一个三层全连接网络（TitanicMLP，继承 torch. nn. Module）：

层	结构	作用
输入层	13 维特征	处理后的特征矩阵
隐藏层 1	Linear(13 → 16) +ReLU	提取特征组合
隐藏层 2	Linear(16 → 8) +ReLU	进一步抽象
输出层	Linear(8 → 1) +Sigmoid	输出 0~1 的生还概率

为什么用这么小的网络？因为数据只有 891 条、13 个特征，属于很小的表格型数据集。这种数据量下堆很深的网络收益极小，反而容易过拟合，所以浅一点的小网络更合适。输出层必须用 Sigmoid，把结果压到 0~1 之间，这样才能当成“概率”来理解，也才能和 0/1 标签一起喂给 BCELoss。

损失函数与优化器

损失函数选 nn. BCELoss（二元交叉熵），它就是专门用来比较“模型输出的概率”和“真实的 0/1 标签”的，概率偏离标签越远，loss 越大。优化器选 Adam，它自适应调整学习率，对新手比较友好，不用手动调太多东西。

主要超参数

超参数	取值	说明
随机种子 SEED	42	保证结果可复现
训练集 : 测试集	713 : 178（8:2）	按 20%切分
训练轮数 EPOCHS	200	每轮把训练集完整过一遍
批大小 BATCH_SIZE	32	每批 32 条，共 23 批
学习率 LR	1e-3	Adam 的常用默认值
L2 正则 weight_decay	1e-4	缓解过拟合
优化器 / 损失函数	Adam/ BCELoss	二分类标准搭配

五、训练过程与结果

训练时每一轮都会做三件事：optimizer. zero_grad() 清空旧梯度 → loss. backward() 反向传播算梯度 → optimizer. step() 更新参数。每轮结束后分别在训练集和测试集上算一次 loss 和准确率记录下来，用来画图。

轮次	训练 loss	测试 loss	训练 acc	测试 acc
1	0.6530	0.6682	0.6227	0.5899
10	0.4549	0.4606	0.8121	0.7978
40	0.3850	0.4372	0.8401	0.8202
100	0.3647	0.4558	0.8415	0.8146
200	0.3526	0.4651	0.8471	0.8034

最终测试集准确率：80. 34%（178 条测试样本）。

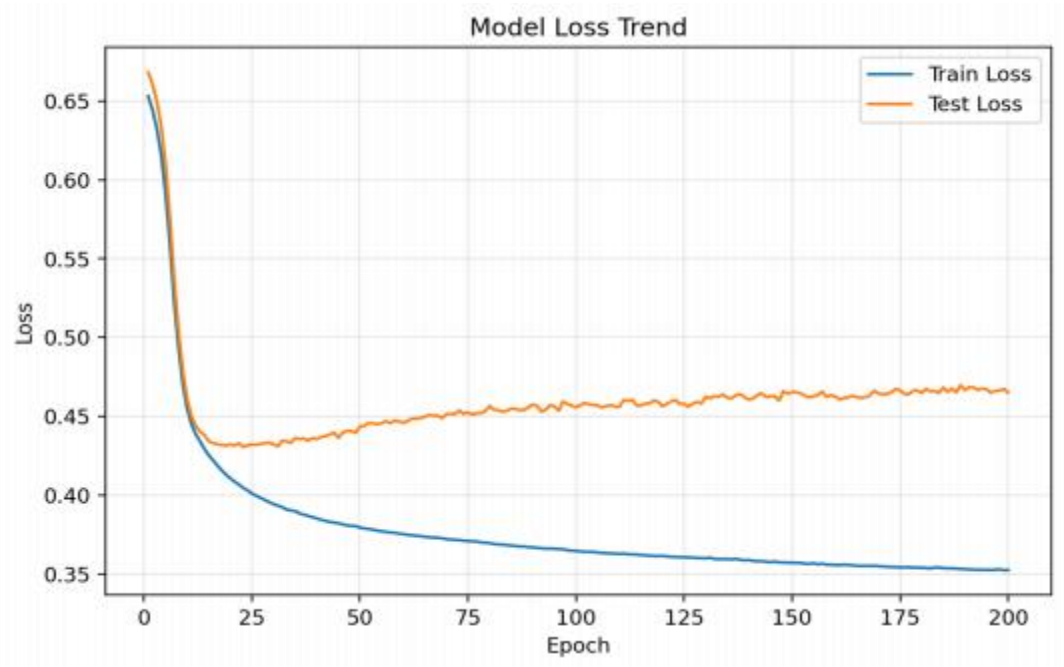


图 1：模型损失趋势图（loss_curve. png）

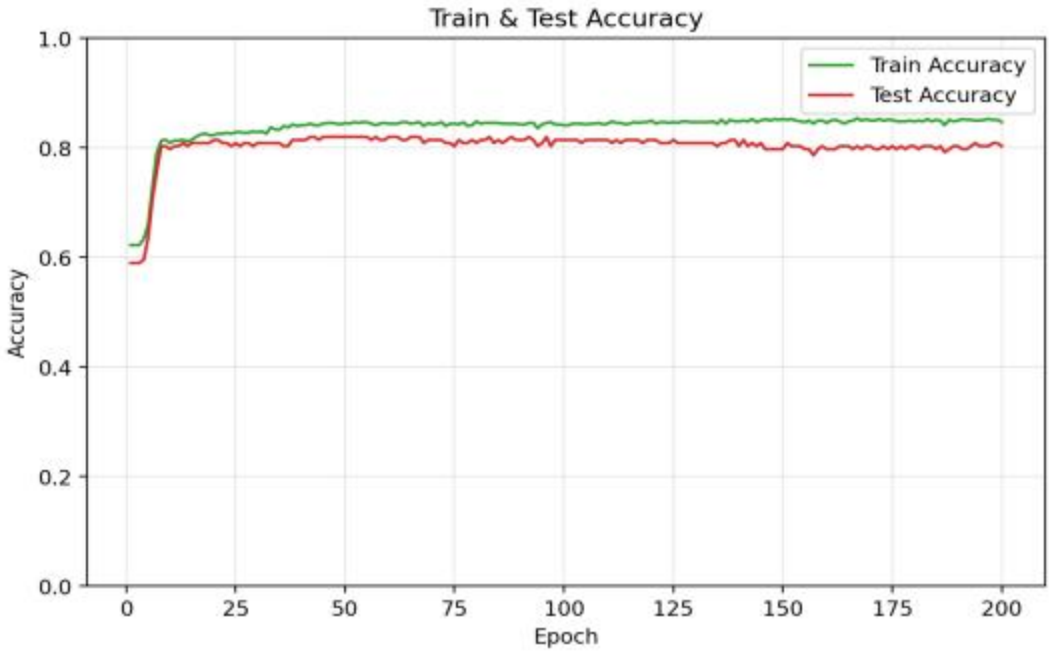


图 2：训练集与测试集准确率（accuracy_curve.png）

六、遇到的问题与解决方法

问题 1：ModuleNotFoundError: No module named'data_process'

卡了最久的一个错。我一开始以为是路径的问题，反复改 sys.path
都没用。后来把目录下的文件名打印出来才发现，文件实际叫 'data_process
.py'——扩展名前面多了一个空格，Python 眼里模块名是带空格的 'data_process '，而 import 语句写的是不带空格的。把文件重命名之后立刻就好了。教训：遇到找不到模块的报错，先确认文件名本身有没有问题，再怀疑路径。

问题 2：特征维度对不上（10 vs 13）

最初 train.py 里把模型输入维度写死成 10，但实际处理出来的特征是 13 维，一跑就会报维度不匹配。改成 input_dim=len(params['feature_cols']) 动态获取之后就正常了，以后增删特征也不用手动改数字。

问题 3：Pclass 被漏掉了

第一版代码里 Pclass 既没放进数值特征也没放进类别特征，等于被悄悄丢掉了，而舱位等级其实是预测生还最强的信息之一。检查特征列表时发现它不在里面，补进类别特征做 one-hot
之后，准确率有肉眼可见的提升。

问题 4：相对路径导致文件存错地方

最早的写法用了 '../docs'、'../output' 这种相对路径，只有在 src
目录下运行才对，换个目录运行就会把文件存到莫名其妙的地方。改成用 __file__
推导项目根目录之后，不管在哪个目录下敲命令都不会出错。

问题 5：轻微的过拟合

从曲线能看出来，大概 40 轮之后训练 loss 还在往下降，但测试 loss 开始慢慢往上走，训练集准确率（84.7%）明显高于测试集（80.3%），这就是典型的过拟合：模型开始死记硬背训练集的细节。我加了 L2
正则化（weight_decay=1e-4）来缓解，测试 loss 略有下降。需要说明的是，训练中途测试准确率最高曾到 82%左右，但我没有去挑那个最高点当作最终结果——因为那是照着测试集的表现选轮次，等于变相让

模型偷看了测试集，评估出来的数字是不可信的。所以这里如实报告最后一轮的 80.34%。

七、结果分析与心得体会

用训练好的模型预测几个新乘客，结果和历史常识是一致的：一等舱女性生还概率 0.97，三等舱中年男性只有 0.11。说明模型确实学到了“妇女儿童优先、舱位越高越容易获救”这类规律,而不是在瞎猜。这也验证了 Sex 和 Pclass 两个特征确实是关键信息。

关于准确率：80%左右对于这个数据集来说是一个正常水平（全猜“遇难”的基线大约是 62%）。我尝试过加 L2 正则，提升很有限，说明瓶颈更多在特征本身而不是模型复杂度——比如 Name 里的称呼（Mr/Mrs/Miss 隐含了性别和年龄信息）、Cabin 的甲板位置其实都还有挖掘空间，但因为缺失太多我暂时放弃了。这也让我理解了题目那句“不要本末倒置”：把精力堆在调参上换一两个百分点，不如把数据处理和特征想清楚。

最大的收获是理解了工程细节和建模同样重要。这次真正卡住我的从来不是网络结构或者数学公式，而是一个文件名里的空格、一个写死的维度数字、一条相对路径。另外，“预处理参数只能在训练集上算”这条规则也非常重要，因为如果把顺序写反、测试集信息会泄露进去。